Software Requirements Specification

for Rentify MERN Web Application

Version 1.0 approved

Prepared by

Utsav Sakariya (23SOECE11033)

Hardip Zapadiya(23SOECE11020)

Bhargav Deshani(23SOEIT11002)

R.K. University

May 2026

Table of Contents

1. Introduction
   1.1 Purpose
   1.2 Document Conventions
   1.3 Intended Audience and Reading Suggestions
   1.4 Project Scope
   1.5 References
2. Overall Description
   2.1 Product Perspective
   2.2 Product Features
   2.3 User Classes and Characteristics
   2.4 Operating Environment
   2.5 Design and Implementation Constraints
   2.6 User Documentation
   2.7 Assumptions and Dependencies
3. System Features
   3.1 User Authentication and Authorization
   3.2 Vehicle Catalog and Browsing
   3.3 Rent Request Processing (Bookings)
   3.4 Payment Processing
   3.5 Administrative Features
4. External Interface Requirements
   4.1 User Interfaces
   4.2 Hardware Interfaces
   4.3 Software Interfaces
   4.4 Communications Interfaces
5. Other Nonfunctional Requirements
   5.1 Performance Requirements
   5.2 Safety Requirements
   5.3 Security Requirements
   5.4 Software Quality Attributes

6. Database & Architecture Models
    6.1 Entity-Relationship (ER) Diagram
    6.2 Data Flow Diagram (DFD)
    6.3 System Architecture
7. Appendix

1. Introduction

1.1 Purpose

The purpose of this document is to define the detailed requirements for the Rentify MERN web application. This comprehensive Software Requirements Specification (SRS) outlines the product's scope, functional and non-functional requirements, technical constraints, and architectural models. It specifies the entire Rentify system, including the Customer Portal, Admin Dashboard, and Database design.

1.2 Document Conventions

- **Bold text** is used to emphasize key terms or feature names.
- Requirements are tagged with unique identifiers (e.g., REQ-1) for traceability.
- Priorities for requirements are indicated as High, Medium, or Low.
- Architectural and Flow diagrams are provided in Section 6.

1.3 Intended Audience and Reading Suggestions

This document is intended for Developers, System Architects, Database Administrators, and Project Managers. The document should be read in sequence, starting from the Overall Description to understand the broad system goals, and referring to the Architectural Models in Section 6 to visualize the data and object flows.

1.4 Project Scope

Rentify is a modern, full-stack MERN (PostgreSQL substituted for MongoDB, paired with Express, React, Node.js) web-based Car Renting Service designed to simplify vehicle rentals. Customers can browse available vehicles, view detailed car information, upload identification documents, manage their profiles, submit rental requests, apply coupons, and process online payments via Razorpay. Administrators have full control over the platform via an Admin Dashboard to manage vehicles, customers, rent requests, coupons, payments, system notifications, maintenance, and expenses for comprehensive business tracking.

1.5 References

- Rentify Project Source Code Repository
- ReactJS and Node.js Documentation
- Sequelize ORM Documentation

2. Overall Description

2.1 Product Perspective

Rentify is a standalone, web-based application built using Node.js, Express.js, React, and PostgreSQL. It follows a decoupled client-server architectural pattern, utilizing RESTful APIs for communication between the React frontend and the Express backend.

2.2 Product Features

- **User Authentication:** Registration, Login, OTP-based Password Reset, and Role-Based Access Control using JWT.
- **Vehicle Management:** Browsing active cars with specifications, pricing, multiple images, and location.
- **Booking Management:** Secure submission of car rental requests, date selection, location selection, and overlap prevention.
- **Customer Profile & Documents:** Updating personal details and uploading Gov ID/License for admin verification.
- **Payment Integration:** Razorpay integration for online payments and refunds.
- **Admin Dashboard:** Centralized control panel with statistical overviews, revenue charts, and business summaries.
- **Entity Management:** CRUD operations on Vehicles, Customers, City Points, Coupons, Maintenance, and Expenses.
- **Review & Contact System:** Customers can leave reviews and contact messages; admins can reply, like, or delete them.

2.3 User Classes and Characteristics

1. **Customer (User):** Public users looking to rent vehicles. They expect an intuitive user interface, accurate vehicle availability, secure payment gateways, and quick booking confirmations.
2. **Administrator:** System managers handling backend operations. They require extensive data tables, bulk operations, reporting tools, business insights (profits/expenses), and direct control over vehicle inventories and approvals.

2.4 Operating Environment

- **Server Platform:** Node.js Runtime Environment.
- **Database:** PostgreSQL.
- **Client:** Modern web browsers (Chrome, Firefox, Safari, Edge).
- **Frameworks:** React (Frontend), Express.js (Backend).

2.5 Design and Implementation Constraints

- The backend must expose RESTful APIs and interact with PostgreSQL using Sequelize ORM.
- File uploads (images, documents) are handled via Cloudinary.
- Email notifications require Nodemailer and an active SMTP service.
- The frontend uses Vite for bundling and Tailwind CSS for styling.

2.6 User Documentation

- `README.md` containing local setup instructions for frontend and backend.

2.7 Assumptions and Dependencies

- Assumes the deployment server has Node.js and PostgreSQL configured.
- Depends on third-party APIs: Razorpay (payments) and Cloudinary (media storage).

3. System Features

3.1 User Authentication and Authorization

3.1.1 Description and Priority

Allow users to securely access the system based on roles (Customer/Admin). **Priority: High**

3.1.2 Stimulus/Response Sequences

- User navigates to `/login` and inputs credentials.
- System validates, generates a JWT, and redirects to the appropriate dashboard.
- Users can request password resets which dispatch a 6-digit OTP via email.

3.2 Vehicle Catalog and Browsing

3.2.1 Description and Priority

Display a searchable catalog of vehicles for customers. **Priority: High**

3.2.2 Stimulus/Response Sequences

- User navigates to `/cars` and applies filters (City, Fuel Type, Transmission, Max Price).
- System queries the PostgreSQL database for active vehicles and populates the view.
- User clicks a car card, navigating to `/cars/:id` to view details and reviews.

3.3 Rent Request Processing (Bookings)

3.3.1 Description and Priority

The core business workflow where a customer requests to rent a car. **Priority: Critical**

3.3.2 Stimulus/Response Sequences

- Customer fills out a booking form on the `/booking` page, selecting dates and locations.
- System checks for date overlaps with existing bookings.
- If valid, the system applies any provided coupons and calculates the final price.
- Admin views the request and can update its status.

3.4 Payment Processing

3.4.1 Description and Priority

Secure handling of rental fees. **Priority: Critical**

3.4.2 Stimulus/Response Sequences

- User selects "Online Payment" for a booking.
- System creates a Razorpay order and opens the payment gateway.
- Upon success, the system verifies the signature and updates the booking to "Paid" and "Confirmed".
- Admins can initiate refunds for cancelled bookings.

3.5 Administrative Features

3.5.1 Description and Priority

Tools for administrators to manage the platform. **Priority: High**

3.5.2 Features Included

- **Business Insights:** View total revenue, GST collected, total expenses, and net profit.
- **Document Verification:** Approve or reject user-uploaded Gov IDs and Licenses.
- **Coupon Distribution:** Send promotional coupon codes to specific or targeted user groups via email.
- **Maintenance Tracking:** Schedule and track car maintenance logs and costs.

4. External Interface Requirements

4.1 User Interfaces

- **Customer UI:** Responsive, component-based design using React and Tailwind CSS. Features dynamic navigation and interactive forms.
- **Admin UI:** Protected layout with a sidebar for quick access to distinct management consoles (Analytics, Cars, Customers, Payments).

4.2 Hardware Interfaces

No specialized hardware needed; relies on standard client devices (PCs, tablets, smartphones).

4.3 Software Interfaces

- **PostgreSQL Database:** Connected via Sequelize ORM (`pg` driver).
- **Cloudinary API:** For image and document uploads (`multer-storage-cloudinary`).
- **Razorpay API:** For processing financial transactions.

4.4 Communications Interfaces

- Standard HTTP/HTTPS protocols for REST API consumption.
- SMTP protocol for Nodemailer email dispatches.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- API response times should generally not exceed 500ms under normal load.
- Images should be optimized by Cloudinary to reduce bandwidth usage.

5.2 Safety Requirements

- Sensitive data (passwords) must be hashed using `bcryptjs` before storage.
- Database backups should be periodically scheduled.

5.3 Security Requirements

- Implement route protection using JWT authorization headers.
- Validate all incoming API requests to prevent SQL Injection and Cross-Site Scripting (XSS).

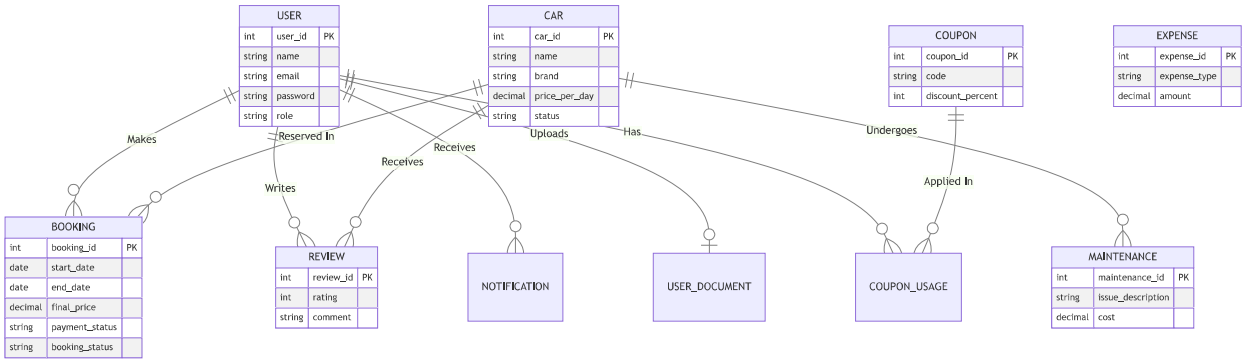
5.4 Software Quality Attributes

- **Maintainability:** Backend routes and controllers must be cleanly separated; frontend components must be reusable.
- **Scalability:** Stateless JWT authentication allows the backend to be horizontally scaled.

6. Database & Architecture Models

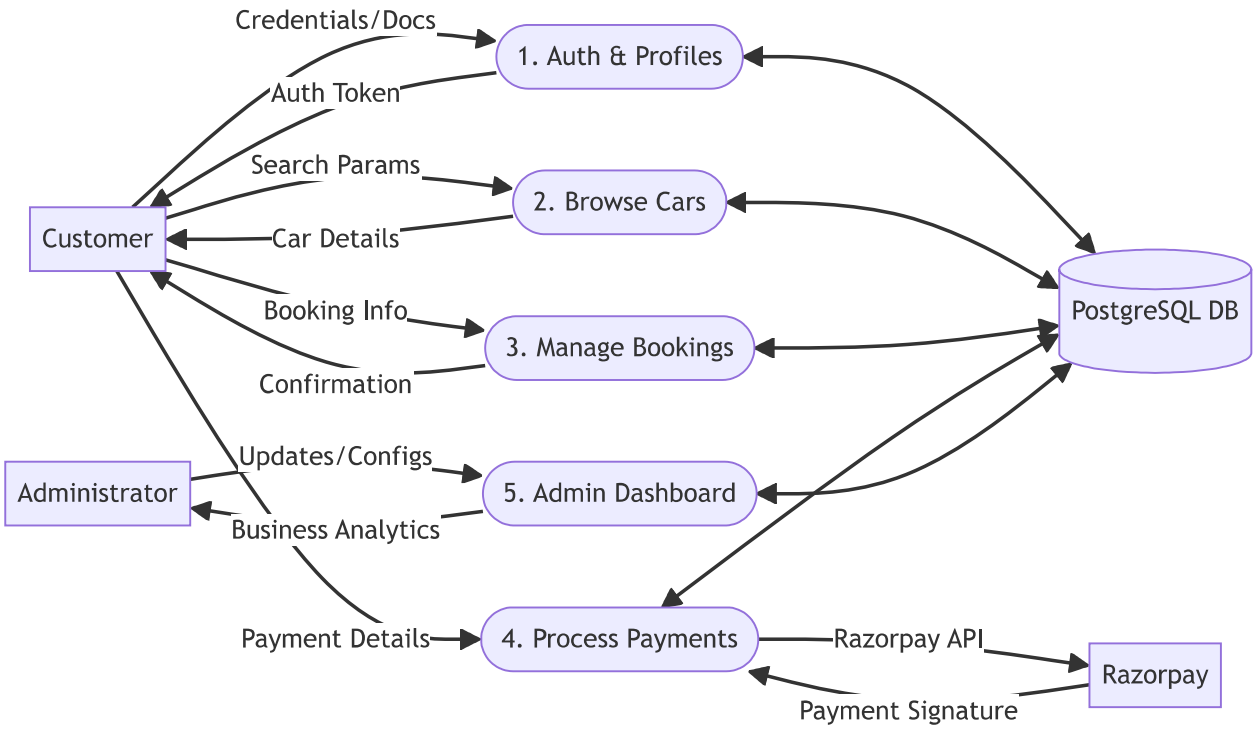
6.1 Entity-Relationship (ER) Diagram

This diagram outlines the relational mapping of the PostgreSQL database tables.



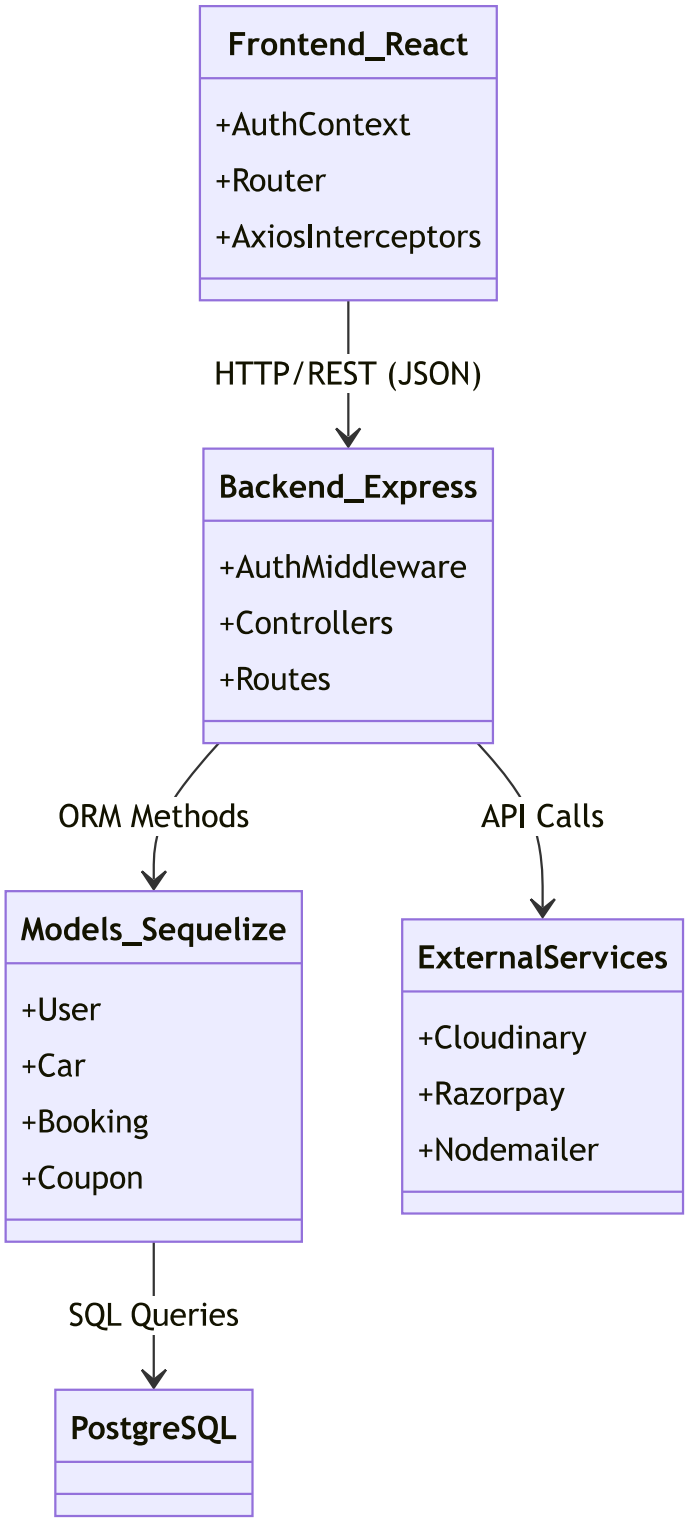
6.2 Data Flow Diagram (DFD)

This diagram illustrates the data flows between external entities and core system processes.



6.3 System Architecture

This class diagram highlights the separation of concerns in the MERN architecture.


Frontend_React
+AuthContext
+Router
+AxiosInterceptors
HTTP/REST (JSON)
Backend_Express
+AuthMiddleware
+Controllers
+Routes
ORM Methods
API Calls
Models_Sequelize
+User
+Car
+Booking
+Coupon
ExternalServices
+Cloudinary
+Razorpay
+Nodemailer
SQL Queries
PostgreSQL

7. Appendix

- **OTP:** One-Time Password used for account recovery.
- **MERN:** MongoDB, Express, React, Node (In this project, PostgreSQL is used instead of MongoDB).
- **JWT:** JSON Web Token used for stateless authentication.